Parallel balanced minimum evolution search in Phylo2Vec coordinates

Abstract

Balanced minimum evolution (BME) provides a distance-based criterion for phylogenetic inference, but searching large tree spaces requires repeated evaluation of many candidate rearrangements. We develop a shared evaluator that computes all valid single-coordinate BME changes for an N-leaf tree in an ordered Phylo2Vec representation in $O(N^2)$ work and supports parallel GPU execution. For larger trees, we optimize the connections between fixed components on a smaller skeleton tree. Balanced averages of the original distances make the skeleton objective equal to the full-tree objective up to a constant. Every skeleton change therefore gives the corresponding full-tree BME change, with an $O(K^2)$ coordinate sweep once the distances between its K components are available. Independent CPU checks agree with direct BME differences to maximum absolute discrepancies of 3.553×10^{-14} for coordinate evaluation and 1.564×10^{-13} for condensation. In 120 small-tree comparisons, the CPU optimizer matches or improves the reference BME score in 119 cases. A full coordinate sweep at $N = 40{,}000$ takes 158.36 ms on eight H100 GPUs. The reduction preserves the specified BME objective. Numerical evaluation uses floating-point arithmetic, and topology search remains heuristic.

1 Introduction

Distance-based phylogenetic inference first estimates pairwise evolutionary distances and then searches for a tree that explains them. Balanced minimum evolution (BME) scores a topology using a weighted sum of those distances, with weights determined by the branching structure along each leaf-to-leaf path [1, 2, 3]. The computational challenge is to evaluate many possible rearrangements repeatedly as the tree changes. Classical subtree-prune-and-regraft (SPR) recurrences already permit quadratic-time neighborhood sweeps [4, 5]. At large scales, their practical cost depends on how calculations are shared, how distance tables are stored, and how work is distributed across processors.

For a tree with N leaves, we organize BME search around a table of coordinate changes in the Phylo2Vec representation. Each entry gives the change in the objective when one integer in the tree's encoding is replaced. The table is evaluated through shared subtree-distance calculations, exposing independent work for parallel execution. Relabeling the current tree changes which rearrangements are represented by single-coordinate updates while preserving the tree and its objective.

The same evaluator operates on a smaller tree when some internal structure is held fixed. We replace each fixed component by one skeleton leaf and retain its contribution through an exact balanced average of the original distances. The full objective then equals the skeleton objective plus a constant that depends only on the fixed components. Consequently, every change evaluated on the skeleton has the same BME change in the full tree. We call this block of component connections a *minibatch*: it restricts the structures being optimized while retaining all leaves in the objective calculation.

We develop three algorithmic contributions:

1. **Shared coordinate evaluation.** All $(N-1)^2$ valid entries are computed in $O(N^2)$ work, matching the size of the explicit table. A map from coordinate values to regraft edges avoids separately scoring candidate trees.

2. **BME preservation in blocks.** The same computation applies to a K-component skeleton in $O(K^2)$ work after its balanced distances are available. The reduction includes the entire complement of a subtree window and specifies the updates required between windows.

3. **Parallel execution.** Independent distance columns and SPR rows support GPU distribution and selection of the best update in each row. Reconstructing rows when needed reduces the balanced-distance table from approximately $4N^2$ to N^2 entries while retaining quadratic sweep work.

The contribution builds on established representation and BME results. Phylo2Vec introduced coordinate differences and ordered relabeling [6, Section 1.3.2]. Penn et al. [7, Lemmas 0.9–0.11] prove that every rooted binary tree has an ordered representation after relabeling. Each coordinate replacement either leaves the tree unchanged or applies a single SPR move (Lemma B.1). Our evaluator retains the quadratic sweep complexity of classical BME SPR calculations. Balanced subtree averages [2, 3] and the fixed-clade reduction of the BME polytope [10, Section 2.4] provide the basis for exact skeleton evaluation. We state the objective identity and prove the conditions needed by the block algorithm.

GradME uses a cubic-time recurrence and automatic differentiation to optimize an expected BME objective over ordered coordinates [7]. Here evaluation is specialized to a discrete tree, including its full coordinate range. Appendix B.5 gives the connection to GradME's vertex derivatives. Parallel taxon-addition BME [8] and learned SPR selection [9] address tree construction and search policy, respectively. Our experiments assess coordinate accuracy, CPU optimization, and GPU sweep throughput.

2 Methods

2.1 Overview

The method takes a tree and a fixed matrix of distances between its leaves. A full-tree sweep computes the objective change for every valid coordinate replacement of the current encoding. A minibatch sweep applies the same evaluator to a skeleton of fixed components (Figure 1). After selecting and accepting a skeleton update, the components are expanded back into the full tree before the next block is constructed. The full-tree case is recovered by taking every component to be a single leaf. Algorithms 1 and 2 specify the complete optimisation procedure in Section 2.6.

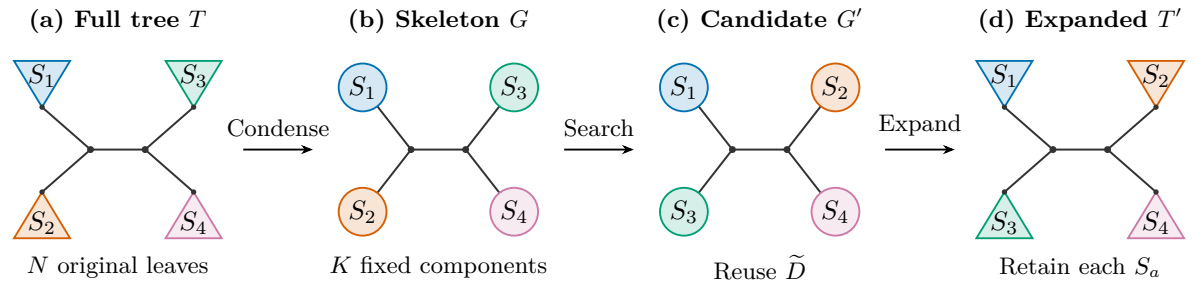
The minibatch reduction preserves full-tree BME score differences in real arithmetic. The distances may be estimates, numerical evaluation uses floating-point arithmetic, and topology search is heuristic.

2.2 BME and Phylo2Vec coordinates

Let T be an unrooted binary tree on $N \geq 3$ labeled leaves, and let D be a fixed, finite, symmetric matrix with zero diagonal. Write $q_T(x, y)$ for the number of internal degree-three vertices on the path from x to y. The BME objective is

$$L(T; D) = \sum_{0 \leq x < y < N} D_{xy}\, 2^{-q_T(x,y)}. \tag{1}$$

If the path has $e_T(x, y)$ edges, then $q_T(x, y) = e_T(x, y) - 1$. Thus the unordered-pair coefficient is $2^{1-e_T(x,y)}$. An artificial degree-two root used to encode the tree is suppressed before evaluating these path lengths. No additivity assumption on D is needed for the identities below.



Balanced component distances $\widetilde{D}_{ab} = \sum_{x \in S_a} \sum_{y \in S_b} \omega_a(x)\omega_b(y) D_{xy} \quad (a \neq b)$

Identical objective changes $L(T'; D) - L(T; D) = L(G'; \widetilde{D}) - L(G; \widetilde{D})$

Figure 1: Block evaluation preserving BME, illustrated with $K = 4$ fixed components. **(a)** Triangles represent components S_a of the full tree, each attached at its apex. **(b)** Each component becomes one skeleton leaf. The condensed distance $\widetilde{D}_{ab}$ retains all leaf pairs between S_a and S_b, with balanced weights ω obtained by halving mass at each binary split. **(c)** Search changes the connections between components while reusing $\widetilde{D}$. **(d)** Expanding the candidate preserves each component's internal topology and attachment root. Within-component contributions cancel, so the skeleton and full-tree objective changes agree. A proper subtree window includes its entire complement as an additional component.

The Phylo2Vec domain for rooted binary trees is

$$\mathcal{V}_N = \{(v_0, \ldots, v_{N-2}) : 0 \leq v_i \leq 2i\}, \qquad |\mathcal{V}_N| = (2N-3)!!. \tag{2}$$

We use the ordered, or cherry-first, subset $\mathcal{V}_N^{\text{cf}} = \{v : 0 \leq v_i \leq i\}$ as the baseline for coordinate evaluation. Every rooted binary tree can be represented in this subset after a suitable leaf permutation [7, Lemmas 0.9–0.11]. Appendix B gives the child-order rule used here. If $\pi(a)$ is the original leaf represented by current index a, the distance matrix in these coordinates is $D_\pi[a, b] = D[\pi(a), \pi(b)]$. Permuting both axes in this way preserves the objective. Constructing the permutation takes $O(N)$ time from a decoded tree, while explicitly copying a dense permuted matrix takes $O(N^2)$ time.

Write $T(v)$ for the unrooted tree obtained by decoding v and suppressing its artificial root. For an ordered baseline v, define the discrete coordinate difference

$$\Delta_{i,j} = L(T(v^{(i \leftarrow j)}); D_\pi) - L(T(v); D_\pi), \qquad 0 \leq j \leq 2i. \tag{3}$$

Here $v^{(i \leftarrow j)}$ replaces only entry i by j. Row i of the table therefore enumerates the allowed replacements of one coordinate, and $\Delta_{i,j} < 0$ identifies an improving move. We use *discrete gradient* as a synonym for this table of finite objective differences, and *coordinate sweep* for one evaluation of the table at a fixed tree. There are $(N-1)^2$ valid coordinate entries, including unchanged coordinates and entries that can represent the same unrooted topology. This count is not a count of distinct SPR neighbors. General candidate values $j > i$ are allowed even though the baseline is ordered. After accepting such a move, the resulting tree is decoded and relabeled before the next ordered evaluation. Relabeling changes computational indices and permutes both

axes of D together, so the observations remain attached to their original leaves.

2.3 Quadratic-time coordinate evaluation

The evaluator first constructs a shared table of subtree distances, then propagates score changes across regraft positions. For distinct vertices u, w, let $A(u, w)$ be the balanced mean distance under D_π between the components rooted at u, w and directed away from their connecting path. If a, b are the two neighbours of an internal vertex u away from w, equal weighting of the two branches gives

$$A(u, w) = \tfrac{1}{2}\{A(a, w) + A(b, w)\}. \tag{4}$$

For a leaf x, initialize $A(y, x) = D_\pi[y, x]$ at the other leaves and apply Equation (4) towards x, processing children before their parent. Each of the $N - 2$ internal entries is one average of two previously computed values, so the entire column costs $O(N)$. Once all leaf columns are complete, symmetry supplies the leaf entries $A(x, w) = A(w, x)$ for each internal target w. The same recurrence computes the entries in its three incident branches in $O(N)$ operations; the diagonal is unused. Thus all $2N - 2$ columns cost $O(N^2)$, including on unbalanced trees. Appendix C describes the implementation with a fixed computational root.

For each coordinate $i \geq 1$, varying v_i regrafts a fixed subtree P_i (Lemma B.1). Moving its attachment between adjacent edges is a nearest-neighbour interchange (NNI). For the four components P, Q, R, S surrounding the current NNI edge, let d_{UV} denote the balanced mean distance between U, V at their attachment roots. The interchange $PQ \mid RS \to PR \mid QS$ changes the BME score by

$$\delta_{\mathrm{NNI}} = \tfrac{1}{4}(d_{PR} + d_{QS} - d_{PQ} - d_{RS}). \tag{5}$$

The standard SPR recurrence accumulates these increments along the remaining tree, obtaining the changing component means from A and a constant number of running averages and weights per traversal state [4]. Each edge therefore costs $O(1)$ operations, and a single traversal scores all regraft positions for P_i in $O(N)$. The edge-to-coordinate map in Appendix B.4 extracts the scores for the $2i + 1$ coordinate values.

Proposition 2.1 (Quadratic coordinate evaluation)**.** *Given an ordered N-leaf encoding v, its leaf permutation π, and constant-time access to D_π, all $(N-1)^2$ entries in Equation (3) can be evaluated using $O(N^2)$ arithmetic operations and $O(N^2)$ storage. In real arithmetic, every entry equals the corresponding full-tree BME difference.*

Proof. Constructing A costs $O(N^2)$. For each row $i = 1, \ldots, N - 2$, the SPR traversal and coordinate mapping take $O(N)$ operations, while $\Delta_{0,0} = 0$. The total is therefore

$$\underbrace{O(N^2)}_{\text{construct } A} + \underbrace{(N-2)\,O(N)}_{\text{evaluate coordinate rows}} = O(N^2).$$

The stored tables have $O(N^2)$ entries. The SPR recurrence gives changes relative to the original attachment, and the edge map in Appendix B.4 identifies them with the corresponding coordinate replacements. □

Every candidate topology is either unchanged or an SPR neighbour; the coordinate table need not cover the full SPR neighbourhood.

2.4 Condensed distances and BME preservation

Partition all N leaves into $K \geq 3$ fixed components $S_1, \ldots, S_K$. Each component has a fixed internal binary topology and a single attachment to the rest of the tree. A component can consist of one leaf. For a non-singleton component, let r_a be the internal vertex adjacent to its attachment

edge, and let $h_a(x)$ count the branching vertices from r_a to leaf x, including r_a. For a singleton, set $h_a(x) = 0$. Define its balanced leaf weights by

$$\omega_a(x) = 2^{-h_a(x)}, \qquad x \in S_a. \tag{6}$$

The weights sum to one within each component (Lemma A.1 in Appendix A).

Let G be an unrooted binary skeleton with one leaf for each component. Construct T_G by replacing skeleton leaf a with S_a, retaining the attachment edge. Define a condensed distance matrix by

$$\widetilde{D}_{ab} = \sum_{x \in S_a} \sum_{y \in S_b} \omega_a(x)\omega_b(y) D_{xy} \quad (a \neq b), \qquad \widetilde{D}_{aa} = 0. \tag{7}$$

These are exact balanced averages of the original pairwise distances. They can be formed by the weighted sum above or, equivalently, by recursively averaging over the two children of each component root. All leaf contributions are retained. This is an objective-level form of the fixed-clade reduction discussed by Haws et al. [10, Section 2.4]. We include the proof to specify the weights and the scope of exactness needed by the algorithm.

Theorem 2.2 (BME decomposition over fixed components)**.** *There is a constant C, independent of the skeleton topology, such that*

$$L(T_G; D) = C + L(G; \widetilde{D}). \tag{8}$$

Consequently, for any two skeletons G and G',

$$L(T_{G'}; D) - L(T_G; D) = L(G'; \widetilde{D}) - L(G; \widetilde{D}). \tag{9}$$

The proof in Appendix A separates the fixed within-component terms from the between-component terms, whose weights factor through the skeleton.

The constant C contains the contributions of pairs within the same fixed component. It cancels when comparing skeletons, so the reduction preserves every objective change even though the skeleton's absolute score omits C. The result applies to every reconnection that preserves the components and to arbitrary fixed dissimilarities, including non-additive estimated distances.

Corollary 2.3 (Exact minibatch coordinate differences)**.** *Let v encode an ordered skeleton G, keep the attached components fixed, and index $\widetilde{D}$ by the same leaf labeling. Evaluating a coordinate replacement $v_i \leftarrow j$ using the exact matrix $\widetilde{D}$ gives*

$$\Delta^{\mathrm{skel}}_{i,j} = L(T_{G(v^{(i \leftarrow j)})}; D) - L(T_{G(v)}; D). \tag{10}$$

Once $\widetilde{D}$ is available, all valid skeleton coordinate differences can be evaluated in $O(K^2)$ time.

Proof. Apply Proposition 2.1 to the K-leaf skeleton with input $\widetilde{D}$. Equation (9) identifies each resulting skeleton difference with the corresponding full-tree difference. □

Corollary 2.4 (A proper subtree window)**.** *For a window attached to the rest of a binary tree by one edge, treat its entire complement as an additional component oriented away from that edge. If its internal topology and exact balanced distances to the other components are retained, Equation (9) applies to rearrangements of the window skeleton.*

Proof. Cutting the attachment edge produces two components. The endpoint in the complement has two outward branches unless the complement is a singleton. It therefore defines precisely a binary component of the type used in the theorem. Including it completes the partition of all leaves, so both within-window and window-to-complement contributions are included. □

The outside component retains distances between leaves inside and outside the window. Deleting the complement would change the objective being optimized. The weights entering the outside distances are determined by the current topology of the complement and its attachment edge. They must be updated when a preceding rearrangement changes that topology.

2.5 Minibatch evaluation and sequential updates

With the fixed components and exact condensed matrix in place, candidate skeletons are evaluated against the full-tree objective through Corollary 2.3. The same matrix can be reused throughout a skeleton search because its component topologies and attachment roots remain fixed. For a candidate G', let $\mathcal{P}(G, G') = \{(a, b) : a < b,\ q_G(a, b) \neq q_{G'}(a, b)\}$. Its full-tree BME change can also be computed directly from the changed pairs:

$$\delta = \sum_{(a,b)\in\mathcal{P}(G,G')} \left(2^{-q_{G'}(a,b)} - 2^{-q_G(a,b)}\right) \tilde{D}_{ab}. \tag{11}$$

Unchanged pairs contribute zero, and the within-component constant cancels. Theorem 2.2 therefore identifies δ with the full-tree BME change. Thus both coordinate selection and the evaluation of a combined candidate can use the same exact condensed distances.

Windows are committed sequentially. After a commit, affected ancestor summaries are updated and the next window's outside distances are constructed from the current tree. Node-disjoint windows need not have additive BME changes because both updates can alter the coefficients of distances between their leaves. For consecutive accepted trees $T^{(0)}, \ldots, T^{(M)}$, exact arithmetic gives

$$L(T^{(M)}; D) - L(T^{(0)}; D) = \sum_{m=1}^{M} \left[L(T^{(m)}; D) - L(T^{(m-1)}; D)\right]. \tag{12}$$

If each increment is negative, so is the cumulative increment. This argument requires that each acceptance calculation uses the current tree, rather than a shared pre-update tree.

On sufficiently deep trees, finite-precision leaf weights can also underflow.

2.6 Optimisation algorithm

Algorithm 1 separates block construction from candidate search. Its inputs are a starting tree T_0, fixed distances D, a schedule $\mathcal{W}$, and a skeleton solver $\mathcal{A}$. Skeleton returns a partition $\mathcal{S}$ of all leaves into fixed rooted components and their current skeleton G. For a proper subtree window, $\mathcal{S}$ includes the entire oriented complement. Weights constructs their balanced weights $\Omega = \{(S_a, \omega_a)\}_{a=1}^{K}$. Condense evaluates Equation (7) without truncation, and Expand restores the same components with their original leaf identities. The components and their attachment roots remain fixed during each solver call. Full-tree search is the special case of one block containing N singleton components.

Algorithm 1 Full-tree and minibatch BME search

Input: T_0, D, schedule $\mathcal{W}$, solver $\mathcal{A}$, block and sweep budgets B, I, margin $\varepsilon \geq 0$.
Output: tree T and accumulated score change s.

		Execution
1	$T \leftarrow T_0$; $s \leftarrow 0$	CPU
2	**for** $b = 0, \ldots, B-1$ **do**	CPU
3	$w \leftarrow \mathcal{W}(T, b)$; **if** $w = \perp$ **then break**	CPU
4	$(G, \mathcal{S}) \leftarrow$ Skeleton(T, w)	CPU
5	$\Omega \leftarrow$ Weights$(\mathcal{S})$	CPU
6	$M \leftarrow$ Condense(Ω, D)	GPU
7	$G' \leftarrow \mathcal{A}(G, M; I)$	CPU+GPU
8	$\delta \leftarrow L(G'; M) - L(G; M)$	CPU+GPU
9	**if** $\delta < -\varepsilon$ **then**	CPU
10	$T \leftarrow$ Expand$(G', \mathcal{S})$; $s \leftarrow s + \delta$	CPU
11	Refresh(T)	CPU
12	**return** (T, s)	CPU

Execution labels refer to the GPU implementation. CPU+GPU denotes a mixed stage.

Each block is constructed from the current tree. Refresh updates affected ancestor summaries, and outside distances are rebuilt before the next window. The acceptance test evaluates the joint candidate, including interactions between its coordinate updates. By Theorem 2.2, every committed change decreases the full-tree objective in real arithmetic. A schedule may expose overlapping local windows and larger backbone connections. Its construction and implementation parameters are specified in Appendix F.1.

Algorithm 2 gives the CPU skeleton solver. The persistent state (v, π) contains the encoding and its map to original component identities, with $M_\pi[a, b] = M[\pi(a), \pi(b)]$. Normalise leaves an ordered encoding unchanged and otherwise relabels it, composing the permutation. Sweep returns the current score and coordinate differences. RankMin sorts the improving row minima. BatchStep tests compatible batches by their joint score, halves a rejected batch, and falls back to the best single move. Best selects the lower-scoring tree, retaining its first argument on a tie.

Algorithm 2 CPU skeleton search

Input: G_0, component matrix M, sweep and escape budgets I, E, margin rule $\epsilon(\ell)$.
Output: best scored skeleton in its original component labeling.

$(v, \pi) \leftarrow \mathsf{Order}(G_0)$; $G_\star \leftarrow G_0$; $r \leftarrow 0$; $e \leftarrow 0$
for $t = 1, \ldots, I$ **do**
  $(v, \pi) \leftarrow \mathsf{Normalise}(v, \pi)$
  $(\ell, \Delta) \leftarrow \mathsf{Sweep}(v, M_\pi)$; $\eta \leftarrow \epsilon(\ell)$
  $G_\star \leftarrow \mathsf{Best}(G_\star, \mathsf{Decode}(v, \pi); M)$
  $R \leftarrow \mathsf{RankMin}(\Delta, -\eta)$
  if $R \neq \varnothing$ **then**
    $v \leftarrow \mathsf{BatchStep}(v, M_\pi, R, \eta)$; $r \leftarrow 0$; **continue**
  $u \leftarrow \mathsf{ExtraSPR}(v, M_\pi, \eta)$
  if $u = \perp$ **and** $r = 0$ **then**
    $(v, \pi) \leftarrow \mathsf{Reverse}(v, \pi)$; $r \leftarrow 1$; **continue**
  if $u = \perp$ **then** $u \leftarrow \mathsf{RemainingSPR}(v, M_\pi, \eta)$
  if $u = \perp$ **and** $e < E$ **and** $t < I$ **then**
    $e \leftarrow e + 1$; $u \leftarrow \mathsf{Escape}(v, \Delta, e, \eta)$
  if $u = \perp$ **then break**
  $v \leftarrow u$; $r \leftarrow 0$
return $\mathsf{Best}(G_\star, \mathsf{Decode}(v, \pi); M)$

Here $\perp$ denotes failure to find a candidate, and r records whether the reversed ordered labeling has been tried since the last topology change. ExtraSPR checks additional regrafts for the pruned components already scanned, while RemainingSPR completes the unrooted SPR scan, including complement-side prunes. Both require a change below $-\eta$. Reverse changes the ordered labeling and composes π without changing the tree. Escape proposes a deterministic pair of uphill coordinate replacements, with the best scored tree retained across such trials. Appendix F.1 defines the batch and escape rules.

The GPU solver is an alternative $\mathcal{A}$. Balanced-distance recurrences, row-minimum evaluation, and score reductions run on the GPU. The CPU packs the SPR proposals, updates and relabels the tree, and retains the best scored skeleton without the CPU completion scan. For the block change, the GPU supplies scores or pair-distance contributions and the CPU assembles the final difference and decides acceptance. The sequence variant's approximate proposal distances are described in Appendix D. The objective-preservation guarantee requires complete distances for acceptance and current outside information. A block or sweep budget limits the search, and only a completed unsuccessful SPR scan establishes SPR-local optimality up to the numerical margin. Neither solver guarantees a global BME optimum.

2.7 Parallel evaluation and computational cost

Equation (4) has an independent recurrence for each fixed column. Assigning columns to threads lets neighboring threads access consecutive entries while following the same traversal order. The top-down correction for column w follows its ancestor path. Depth-first entry intervals determine the next child without a traversal stack. Its total work over columns is $O(\sum_w \text{depth}(w))$, bounded by $O(N^2)$ and reduced to $O(N \log N)$ for trees of logarithmic height.

A full balanced-distance table stores approximately $4N^2$ entries. The column-tiled variant retains the internal-to-leaf block and reconstructs required internal rows on demand. This reduces that table to approximately N^2 entries, while the input matrix and other buffers still contribute to total memory. A row-wise reduction can return the best coordinate replacement in each row without storing the full coordinate table. Columns and SPR rows can also be distributed across GPUs. Appendix C gives the dependency and workspace arguments.

For a skeleton with K leaves, a coordinate sweep costs $O(K^2)$ once its distances are available. This does not make arbitrary distance construction independent of N. Direct evaluation of one condensed pair can require $|S_a||S_b|$ original distances. Explicit leaf weights use $O(N)$ entries, and constructing exact condensed distances can dominate runtime for large component pairs. When $D_{xy} = \phi(x)^\top \psi(y)$ has a fixed-dimensional bilinear representation, weighted component embeddings give $\widetilde{D}_{ab} = (\sum_x \omega_a(x)\phi(x))^\top (\sum_y \omega_b(y)\psi(y))$. That shortcut is exact for the specified bilinear distance. It does not justify averaging embeddings before applying a nonlinear sequence-distance correction.

3 Experimental design

The evaluation addresses three questions: whether computed changes agree with the BME objective, how the CPU optimizer behaves from a common starting tree, and how coordinate sweeps scale on GPUs. Table 1 identifies the computation measured by each experiment.

Table 1: Scope of the experiments. A coordinate sweep evaluates candidates at one fixed tree. Optimization includes successive accepted updates.

Experiment	Computation	Evidence supplied
Objective checks	Coordinate and condensed changes against direct BME	Numerical agreement on independently evaluated cases
CPU optimization	Search from identical starting trees and distance matrices	Final objective, topology, and optimization time
GPU sweeps	Full-tree coordinate evaluation	Evaluation time and memory at a fixed tree
Embedding blocks	Exact bilinear condensation and skeleton sweeps	Cost of these two stages under a bilinear distance
Sequence variant (Appendix D)	Approximate proposals followed by full-objective acceptance	Refinement time for that proposal-and-check procedure

The CPU comparison uses additive distances from trees whose branch lengths are uniform on $(0.05, 0.55)$ and a noisy variant $D'_{xy} = D_{xy} \exp(0.15 Z_{xy})$, with symmetric standard-normal Z_{xy}. There are respectively 20, 20, 10, 5, and 5 replicates at $N = 20, 50, 100, 150, 200$ in each regime. The reference is our in-memory C implementation of the FastME BME-SPR algorithm. Agreement with the external FastME executable is assessed separately. A score tolerance of 10^{-9} defines wins, ties, and losses. Single-core timings are reported for Apple M5 and AMD Threadripper PRO 5995WX processors.

GPU evaluation uses NVIDIA RTX A6000 and H100 devices, with additional CPU/Metal measurements in the supplement. The GPU coordinate path uses FP32 working arrays in the reported comparisons, whereas independent score calculations use FP64. The embedding workload

Table 2: Agreement with independent BME calculations. Errors are maximum absolute discrepancies over the stated tests.

Calculation	Test scope	Maximum error
CPU coordinate differences	36,000 changes, $N = 20$	3.553×10^{-14}
Ordered-tree enumeration	28,488 changes, $N = 4$–7	2.776×10^{-15}
Relabeling, first orientation	200 trees, $N = 25$	1.776×10^{-14}
Relabeling, reversed orientation	Same 200 trees	2.665×10^{-14}
Condensed SPR differences	16,231 changes, $N = 40$, $K = 16$	1.564×10^{-13}
FP32 CUDA gradient	Reported sizes $N = 100$–$4{,}000$	1.11×10^{-6}

constructs condensed distances through the bilinear factorization in Section 2.7. The sequence variant instead uses approximate condensed distances for proposals and full-tree calculations for acceptance, as detailed in Appendix D. Its timings measure that procedure rather than exact condensed-distance construction for every proposal.

We assess topological error by counting inferred nontrivial splits absent from the reference tree. For binary trees on the same leaves, normalized Robinson–Foulds (RF) distance is this count divided by $N - 3$.

4 Results

4.1 Agreement with independent objective calculations

Table 2 compares coordinate and condensed changes with independent objective calculations. The largest discrepancy in the CPU coordinate tests is 3.553×10^{-14}, and the condensation tests agree to 1.564×10^{-13}. These are observed absolute errors on the tested cases, not universal error bounds. Rooted-vector round trips recover all 1,000 tested vectors at $N = 40$. Unrooted round trips preserve the split sets in a further 1,000 cases. Exhaustive rooted-tree enumeration at $N = 4, \ldots, 7$ checks 11,460 encodings.

Independent enumeration also identifies the scope of the coordinate neighborhood. On 30 random trees at each tested size from $N = 8$ to 20, a single ordered orientation covers approximately 27–33% of distinct SPR neighbors. Two orientations together cover approximately 52–56%. Neither constitutes an exhaustive SPR search. The full CPU optimizer's additional SPR scan is therefore material: independent enumeration found no improving SPR move in 90 returned trees at $N = 10, 16, 24$. This observation concerns that tested configuration, not the minibatch variant.

4.2 CPU optimization

Both methods recover the reference topology in all 60 noiseless trials. On the 60 noisy trials, Phylo2Vec obtains two lower BME scores, 57 ties, and one higher score than the in-memory reference. Thus it matches or improves the reference objective in 119 of 120 cases overall. A better BME score need not imply a smaller RF distance when the input distances are noisy. For example, the $N = 200$ noisy comparison has a lower mean reference RF error despite one BME-score win for Phylo2Vec.

At $N = 200$, the noiseless Apple M5 comparison requires 63.6 coordinate sweeps on average, versus 211.0 reference SPR iterations. The corresponding mean times are 18.14 and 75.46 ms. Table 3 gives the largest-size timings for both processors. Supplementary Tables 5 and 6 give results at all sizes. External FastME comparisons at the same five sizes produce identical final unrooted splits and BME differences below 10^{-8} in the reported checks. These checks support the reference implementation on those inputs without making the in-memory timings measurements of the external executable.

Table 3: Single-core optimization at $N = 200$. Times are mean ± standard deviation in milliseconds over five replicates.

Processor	Distances	Reference	Phylo2Vec	Ratio
Apple M5	Additive	75.46 ± 2.42	18.14 ± 0.74	4.16
Apple M5	Noisy	85.16 ± 2.06	24.67 ± 2.79	3.45
AMD 5995WX	Additive	160.64 ± 5.51	27.96 ± 1.18	5.74
AMD 5995WX	Noisy	181.97 ± 4.30	36.91 ± 3.94	4.93

4.3 Coordinate sweep throughput and memory

At $N = 4{,}000$, a full coordinate sweep takes 6.219 ms on one H100, compared with 701.375 ms for its CPU reference. At $N = 40{,}000$, distributing the full-table computation across eight H100s reduces time from 816.34 to 158.36 ms, a factor of 5.15. The SPR stage alone falls from 756.26 to 98.28 ms. Column tiling reduces storage at the cost of recomputation: the reported one-GPU memory requirement falls from 31,142 to 13,614 MB, while the eight-GPU tiled evaluation takes 1,032.83 ms at this size. Thus the reduced-memory variant is useful when capacity, rather than latency, is limiting.

Distributed and streamed configurations extend a single full-tree sweep to $N = 380{,}000$, taking 205.450 s with 72,946 MB reported per GPU on eight H100s. All these times measure evaluation at a fixed tree.

For the separate embedding workload with $N = 10{,}000{,}000$ and $K = 20{,}000$, average condensation and coordinate-sweep times are 10.20 and 48.57 ms per batch on eight H100s. The reported epoch estimate, 39.20 s, is the sum of those stage times multiplied by 667 batches. It measures the projected cost of those stages over the prescribed blocks, excluding additional search work and optimization to convergence. The complete scaling tables are in Appendix H.

5 Discussion

The method separates the size of a search block from the objective it evaluates. A skeleton can optimize connections among a few components while retaining the contribution of every original leaf through exact condensed distances. The same coordinate evaluator therefore supports both full-tree sweeps and restricted block searches. This links the established BME component identity to a common computational interface for parallel tree search.

The main computational tradeoff is between reuse and reconstruction. Dense tables permit rapid sweeps but require quadratic memory. Column tiling trades memory for repeated arithmetic. Skeletons reduce the dimension of the optimization problem, but exact construction of their distances can retain a dependence on the full leaf set. This becomes particularly visible in nonlinear sequence-distance refinement and in candidates involving large condensed components. Bilinear embedding benchmarks therefore establish a different scaling regime from the sequence experiments.

The experiments support faster coordinate evaluation and competitive CPU optimization on the tested instances. Their scope leaves three limitations. First, the tested relabeling and block schedules do not establish exhaustive coverage or global convergence, and the total number of sweeps remains instance-dependent. Second, the numerical checks give observed discrepancies rather than universal error bounds, and a lower BME score under estimated distances need not imply a more accurate topology. Third, the large sequence comparisons use approximate proposals with full-objective acceptance and do not report replicate uncertainty at the largest sizes. Those measurements do not establish the end-to-end cost of a sequence search using exact condensed distances for every proposal.

6 Conclusion

Shared coordinate evaluation and exact component reduction give a common basis for full-tree and block BME search. A quadratic sweep evaluates all valid coordinate updates, and the same computation on an exactly condensed skeleton preserves their full-tree objective changes. The resulting parallel algorithms let a search trade the scope of its candidate set against evaluation cost while retaining the specified BME objective.

References

[1] Y. Pauplin. Direct calculation of a tree length using a distance matrix. *Journal of Molecular Evolution*, 51(1):41–47, 2000.

[2] R. Desper and O. Gascuel. Fast and accurate phylogeny reconstruction algorithms based on the minimum-evolution principle. *Journal of Computational Biology*, 9(5):687–705, 2002.

[3] R. Desper and O. Gascuel. Theoretical foundation of the balanced minimum evolution method of phylogenetic inference and its relationship to weighted least-squares tree fitting. *Molecular Biology and Evolution*, 21(3):587–598, 2004. doi:10.1093/molbev/msh049.

[4] W. Hordijk and O. Gascuel. Improving the efficiency of SPR moves in phylogenetic tree search methods based on maximum likelihood. *Bioinformatics*, 21(24):4338–4347, 2005. doi:10.1093/bioinformatics/bti713.

[5] V. Lefort, R. Desper, and O. Gascuel. FastME 2.0: A comprehensive, accurate, and fast distance-based phylogeny inference program. *Molecular Biology and Evolution*, 32(10):2798–2800, 2015.

[6] M. J. Penn, N. Scheidwasser, M. P. Khurana, D. A. Duchêne, C. A. Donnelly, and S. Bhatt. Phylo2Vec: a vector representation for binary trees. *Systematic Biology*, 74(2):250–266, 2025. Advance access 2024. doi:10.1093/sysbio/syae030.

[7] M. J. Penn, N. Scheidwasser, J. Penn, C. A. Donnelly, D. A. Duchêne, and S. Bhatt. Leaping through tree space: continuous phylogenetic inference for rooted and unrooted trees. *Genome Biology and Evolution*, 15(12):evad213, 2023. doi:10.1093/gbe/evad213. Lemma numbers refer to the appendix of arXiv:2306.05739v4.

[8] C. Tapo and Q. Zhu. Parallel and scalable balanced minimum evolution algorithm for large-scale phylogenetic inference. *bioRxiv*, 2026. Preprint. doi:10.64898/2026.07.27.740982.

[9] F. J. Camerota Verdù, A. Gasparin, L. Bortolussi, and L. Castelli. Learning to explore tree neighbourhoods for phylogenetic inference. *Briefings in Bioinformatics*, 27(1):bbaf732, 2026. doi:10.1093/bib/bbaf732.

[10] D. C. Haws, T. L. Hodge, and R. Yoshida. Optimality of the neighbor joining algorithm and faces of the balanced minimum evolution polytope. *Bulletin of Mathematical Biology*, 73(11):2627–2648, 2011. doi:10.1007/s11538-011-9640-x.

[11] C. Piñeiro, J. M. Abuín, and J. C. Pichel. VeryFastTree: speeding up the estimation of phylogenies for large alignments through parallelization and vectorization strategies. *Bioinformatics*, 36(17):4658–4659, 2020.

[12] S. Mirarab, N. Nguyen, S. Guo, L.-S. Wang, J. Kim, and T. Warnow. PASTA: Ultra-large multiple sequence alignment for nucleotide and amino-acid sequences. *Journal of Computational Biology*, 22(5):377–386, 2015.

[13] M. J. Penn, N. Scheidwasser, C. A. Donnelly, D. A. Duchêne, and S. Bhatt. Bayesian inference of phylogenetic distances: revisiting the eigenvalue approach. *Bulletin of Mathematical Biology*, 87:32, 2025.

A Proof of the exact block reduction

We use the component weights and attachment convention from Section 2.4.

Lemma A.1 (Normalization of leaf weights)**.** *For each component,* $\sum_{x \in S_a} \omega_a(x) = 1$.

Proof. A singleton has weight one. At a binary root, every leaf weight in each child component is multiplied by $1/2$. Induction over the component therefore gives total mass $\frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 1 = 1$. □

A.1 Proof of Theorem 2.2

Proof. Separate the unordered leaf pairs into pairs within a component and pairs between components. For $x, y \in S_a$, the path stays inside S_a and is unchanged by rearranging the skeleton. The sum of all such terms is

$$C = \sum_a \sum_{\substack{x<y \\ x,y \in S_a}} D_{xy}\, 2^{-q_{T_G}(x,y)},$$

which is independent of G.

For $x \in S_a$ and $y \in S_b$ with $a \neq b$, the path contains the branching vertices inside S_a, those on the skeleton path, and those inside S_b. These sets of vertices are disjoint, giving

$$q_{T_G}(x, y) = h_a(x) + q_G(a, b) + h_b(y).$$

Therefore its weight factors as $2^{-q_{T_G}(x,y)} = \omega_a(x)\, 2^{-q_G(a,b)}\, \omega_b(y)$. Summing between-component terms gives

$$\begin{aligned} \sum_{a<b} \sum_{\substack{x \in S_a \\ y \in S_b}} D_{xy}\, 2^{-q_{T_G}(x,y)} &= \sum_{a<b} 2^{-q_G(a,b)} \sum_{\substack{x \in S_a \\ y \in S_b}} \omega_a(x)\omega_b(y) D_{xy} \\ &= L(G; \widetilde{D}). \end{aligned}$$

Adding the within-component constant proves Equation (8). Subtracting two such decompositions cancels C and proves Equation (9). □

B Ordered representation and coordinate mapping

B.1 Linear-time decoding

For $v_i \leq i$, initialize a component-root array by $r[x] = x$. Process $i = N-2, \ldots, 0$. At step i, join the components with roots $r[v_i]$ and $r[i+1]$ at a new node $m = 2N - 2 - i$, then replace $r[v_i]$ by m. Before this step, the active components have distinct minimum labels $0, \ldots, i+1$. Joining the two specified components removes minimum label $i+1$ and preserves minimum label v_i. Induction therefore establishes the invariant and the correctness of the construction. Each step uses constant work, giving an $O(N)$ decoder. This argument depends on the ordered condition and does not apply unchanged to a general vector with $v_i > i$.

B.2 Relabeling a decoded tree

Relabeling uses the constructive labeling of Penn et al. [7, Lemmas 0.9–0.11]. The root receives label zero, and a counter starts at $t = 1$. Internal nodes are visited parent before child. Each node passes its label to one child, gives the other child label t, records the inherited label as v_{t-1}, and increments t. Penn et al. prove that the leaf labels form a permutation of $0, \ldots, N-1$, that the recorded vector is ordered, and that it decodes to the original rooted tree. Their argument holds for either choice of inheriting child at every node. Each vertex is processed a constant number of times, so the construction takes $O(N)$ time. Permuting both axes of D by the inverse leaf-label map preserves every term in Equation (1), so the BME score is unchanged.

The implementation lets the child containing the smaller minimum leaf index in the current labeling inherit, and a second orientation reverses that choice. These are two deterministic coordinate systems for the same current tree. Either gives representational coverage after relabeling, not coverage by a particular optimizer trajectory. For general candidate vectors, the implementation also uses a tree-to-vector encoder. Its unrooted version first inserts a degree-two root on the edge incident to leaf zero, so round-trip correctness is assessed by unrooted splits rather than root placement.

B.3 Coordinate replacements are SPR moves

For an ordered v and a leaf $\ell \geq i+1$, follow the chain $\ell_0 = \ell$, $\ell_{s+1} = v_{\ell_s - 1}$ while $\ell_s > i+1$. Each step uses a coordinate with index greater than i, and the chain strictly decreases. Let P_i be the set of leaves whose chain stops at $i+1$; it contains $i+1$ and does not depend on v_i.

Lemma B.1 (Coordinate neighbors are SPR neighbors)**.** *Let $v \in \mathcal{V}_N^{\mathrm{cf}}$, $0 \leq i \leq N-2$, and $0 \leq j \leq 2i$. Then $T(v^{(i \leftarrow j)})$ either equals $T(v)$ or is obtained from $T(v)$ by one SPR move that prunes the subtree on the leaves P_i.*

Proof. Read $w = v^{(i \leftarrow j)}$ by leaf addition [6]. Starting from leaf 0, coordinate w_k inserts leaf $k+1$ on one of the $2k+1$ edges of the current rooted tree on leaves $0, \ldots, k$, including the edge above its root, and a value $w_k \leq k$ selects the pendant edge of leaf w_k. Only step i depends on j. It inserts leaf $i+1$ on some edge, creating a vertex p. Every later step $k > i$ has $w_k = v_k \leq k$ and inserts leaf $k+1$ on the pendant edge of leaf v_k. The new leaf therefore lies on the same side of p as leaf v_k. By induction on k, the leaves on the side of p that contains $i+1$ are exactly P_i. The insertions among these leaves build a subtree whose topology does not depend on j. Removing P_i and suppressing p leaves the tree T^- produced by steps $0, \ldots, i-1$ and the insertions of leaves outside P_i, which also does not depend on j. Thus every $T(w)$ is T^- with the fixed subtree on P_i attached at one edge of the unrooted T^-. If leaf $i+1$ is inserted above the root, that edge is the one through the root of T^-, which becomes a single edge when the artificial root is suppressed. Taking $j = v_i$ recovers $T(v)$. Two such trees differ only in the attachment edge of P_i, so they are equal or related by one SPR move. □

The lemma bounds the coordinate neighborhood by the SPR neighborhood. It does not show the reverse inclusion. The mapping below identifies the attachment edge for each j.

B.4 Mapping coordinate values to regraft edges

The ordered decoder numbers leaves $0, \ldots, N-1$ and internal nodes $N, \ldots, 2N-2$. The last index is the artificial root. Suppressing it leaves a computational root $R = 2N-3$ and the top edge joining R to the artificial root's other child, denoted c_{other}. Index ordinary edges by their child endpoints.

For row i, let $E_i[u]$ be the score change from pruning the component associated with leaf $i+1$ and regrafting it on the edge indexed by u. The half-tree SPR traversal supplies these values. Let $b_i(j)$ be the root of the component containing leaf j after the merges with coordinate index greater than i. Then the edge-to-coordinate map is

$$\Delta_{i,j} = \begin{cases} E_i[b_i(j)], & 0 \leq j \leq i, \\ E_i[(2N-2) - 2i + j], & i+1 \leq j \leq 2i-2, \\ E_i[c_{\mathrm{other}}], & j \in \{2i-1, 2i\} \cap [i+1, 2i]. \end{cases} \tag{13}$$

The first case identifies the component on the corresponding pendant edge of the prefix tree. In the ordered decoder's node numbering, the remaining non-root prefix edges occupy consecutive indices $2N-1-i, \ldots, 2N-4$, giving the affine map in the second case. The last two rooted

edge choices collapse onto the same unrooted top edge. At $i = 1$, the middle case is empty and $j = 1$ belongs to the first case. Only $j = 2$ takes the collapsed-root value. Coordinate zero has no nontrivial alternative.

On a CPU, the array b_i can be maintained incrementally as the merge order advances. On a GPU, independent rows reconstruct their mappings using the latest eligible merge for each minimum leaf. When several merges have the same minimum leaf, only the latest eligible merge may write that destination. This avoids concurrent writes that would otherwise depend on thread scheduling.

B.5 Connection to continuous vertex derivatives

GradME defines an expected objective $F(W) = \mathbb{E}[f(V)]$ for independent ordered coordinates [7]. For comparison, take $f(v) = L(T(v); D_\pi)$ with the same unrooted convention as Equation (1). Let W^v put probability one on each entry of an ordered vector v. For $j \le i$, change only row i to $(1-t)e_{v_i} + te_j$, where e_j is a unit vector. Only two vectors then have nonzero probability, giving

$$F(W(t)) = (1-t)f(v) + tf(v^{(i \leftarrow j)}), \qquad \left.\frac{dF(W(t))}{dt}\right|_{t=0} = \Delta_{i,j}. \tag{14}$$

This is a direct consequence of the expected-objective definition, not a new exactness property. The computational distinction is the shared quadratic evaluator at a discrete tree. It also evaluates $j > i$, outside the ordered domain of this relaxation. It does not evaluate gradients at arbitrary interior W in quadratic time.

C Parallel dependencies and workspace

C.1 Column ownership and ancestor traversal

For a fixed column w, the bottom-up update in Equation (4) reads only earlier entries of that column. Thus the recurrence has no cross-column dependency once its initial entries are available. The implementation separates the necessary transpose from the recurrence rather than writing both triangular copies concurrently. For a non-root ancestor u of w, let $\mathrm{sib}_u(w)$ be the child of u whose subtree does not contain w. The component at u facing away from w combines this child with the parent direction:

$$A(u, w) = \tfrac{1}{2}\{A(\mathrm{sib}_u(w), w) + A(\mathrm{parent}(u), w)\}. \tag{15}$$

Top-down correction visits ancestors of w in order from the computational root. If a node's children have contiguous depth-first intervals, comparing the entry time of w with their interval boundary selects the child containing w. Only the current node and the fixed target are required as traversal state. The initial degree-three root update uses the other two root branches.

C.2 Autonomous row reconstruction

Store the balanced distances from internal vertices to leaves. For a requested row w, these entries supply its leaf values. For leaf w, the leaf values instead come directly from D. Average child entries bottom-up to fill internal positions, then overwrite ancestors of w top-down using the sibling and parent directions. Initialize the degree-three root correction explicitly and set the diagonal to zero.

For an internal position u in a disjoint component, the bottom-up average is the required balanced distance by linearity. For a descendant u of w, the stored leaf values already use the component at w directed away from that descendant, so averaging within the descendant gives the correct value. Ancestors of w require the complementary direction and are supplied by the top-down overwrite. These cases exhaust the row. The reconstruction costs $O(N + \mathrm{depth}(w)) = O(N)$.

Reconstructing a constant number of rows per pruned component therefore retains quadratic sweep complexity. The approximately fourfold reduction concerns the balanced-distance table only.

C.3 Traversal stack size

When a traversal reaches two outward components, push the larger component first and the smaller second on a last-in-first-out stack. The smaller component is processed next. Each extra pending larger component is associated with descent into a component at most half the size of its predecessor. Hence the number of simultaneously pending components is logarithmic in the number of tree vertices. Including the active traversal state gives a safe bound $\lfloor \log_2(2N-2) \rfloor + 2$. This is a workspace bound, not a statement that the traversal visits fewer vertices.

D Sequence-refinement variant and recorded benchmarks

D.1 Implementation used for the recorded measurements

The sequence implementation used for the measurements in this appendix differs from the exact-distance search described in Section 2.6. For trees larger than 2,048 leaves, it caps the weighted leaf lists used for proposal distances at 64 entries per component. These lists are called profiles in the implementation. Above the cap, unresolved subtree mass is assigned to representative leaves, producing an approximate matrix $\widetilde{D}^{\mathrm{cap}}$. For smaller trees, the proposal calculation is uncapped.

The approximate matrix guides the inner skeleton search. Before accepting an update, the implementation recomputes affected condensed distances involving truncated components using every leaf contribution. Distances for untruncated pairs can be reused. It then evaluates Equation (11), with FP64 accumulation and a numerical acceptance margin of 10^{-6}. Outside weights are rebuilt from the current tree before each window, and ancestor summaries are updated after a commit. The acceptance calculation therefore evaluates the full-tree BME change, subject to rounding. It does not make the earlier proposal gradients exact, and the numerical margin is not a proved floating-point error bound.

These measurements characterize that implementation variant. They are not timings for constructing exact condensed distances and using them throughout every minibatch search.

D.2 Sequence data and comparison protocol

The recorded comparisons use the RNASim benchmark associated with PASTA [12], with simulated sequences and a known reference tree. The reported alignments have 1,596 columns. The minibatch comparison uses gap-aware JC69 distances. Selected full-tree experiments also use the eigenvalue-based distance approach [13], which changes the input objective and is identified separately. The CPU tools are 64-thread FastME 2.1.6 and VeryFastTree 4.0.5 [5, 11]. VeryFastTree's minimum-evolution refinement is run without its likelihood-refinement phase. Refinement time is reported separately from starting-tree construction. The minibatch times include the final full-tree acceptance calculation and sequential maintenance of the outside distances.

D.3 Archived refinement comparison

Table 4 retains the archived comparison of this minibatch variant with VeryFastTree refinement from the same initial topology. These measurements precede the subsequent acceptance-kernel optimizations and characterize that recorded implementation. At $N = 50{,}000$, the minibatch method returns 4,950 incorrect splits in 16.84 s, compared with 4,980 in 313.53 s. At $N = 100{,}000$, the corresponding results are 8,802 in 104.45 s and 8,857 in 593.50 s. The refinement-time ratios are 18.6 and 5.7 respectively. These exclude the shared initialization, which takes 63.01 and

Table 4: Archived RNASim refinement comparison, preceding the later acceptance-kernel optimizations. Both methods use the same starting topology. VFT denotes 64-thread VeryFastTree minimum-evolution refinement. P2V denotes the sequence variant on one RTX A6000, using capped proposal distances above 2,048 leaves and full-tree acceptance checks. Times exclude the shared initialization and characterize this proposal-and-check variant.

N	Initial	Wrong splits		Refinement (s)		Init. (s)
	wrong splits	VFT	P2V	VFT	P2V	
1,000	214	178	176	6.88	0.06	0.58
5,000	890	703	696	33.38	0.68	3.40
10,000	1,667	1,313	1,320	65.98	1.55	7.82
50,000	6,729	4,980	4,950	313.53	16.84	63.01
100,000	12,375	8,857	8,802	593.50	104.45	179.03

179.03 s. Adding that initialization to both methods gives approximately 4.72-fold and 2.73-fold ratios, rather than the refinement-only ratios.

The topological advantage is not uniform. At $N = 10{,}000$, the minibatch result has 1,320 incorrect splits, compared with 1,313 for VeryFastTree. A full-tree run with eigenvalue-based distances reaches 1,309 in 4.57 s, but uses a different distance estimate. The full-tree JC69 run returns 1,320 in 3.71 s. These variants should therefore not be combined into a single claim of consistently better topological accuracy.

At $N = 1{,}000$, FastME applied to the GPU-computed JC69 distance matrix returns 177 incorrect splits in 21.68 s with its own BIONJ initialization. Its shared-start refinement returns 178 in 11.27 s. The corresponding full-tree GPU result is 176 in 0.01 s, and minibatch refinement returns 176 in 0.06 s. The experiments differ in initialization and timing scope, so the ratios between their raw elapsed times are not interchangeable.

E Truncation and stale outside-distance counterexamples

Two constructions illustrate the need for exact condensed distances and current outside information. The first concerns the approximate-proposal variant in Appendix D. On a 4,102-leaf tree containing a balanced 4,096-leaf component, an adversarial split distance can give a capped proposal change of -0.25 while its true full-profile change is $+0.2421875$ for a cap of 64, or $+0.234375$ for a cap of 128. The proposal sign alone is therefore insufficient. Full-profile acceptance rejects both changes.

Separately, consider two disjoint windows whose internal rearrangements change their root-to-leaf weight vectors from p_A, p_B to $p_A + \delta p_A, p_B + \delta p_B$. With a fixed path coefficient κ between their attachment points, their cross contribution contains $\kappa p_A^\top D_{AB} p_B$. The jointly changed contribution includes the additional term

$$\kappa\, \delta p_A^\top D_{AB} \delta p_B,$$

which is absent from the sum of two changes computed against the original surroundings. In a 1,540-leaf test with two 768-leaf windows, changes -0.0032945313 and -0.0254489835 computed against frozen surroundings combine to an actual increase of $+0.0505258348$. Rebuilding the outside profile after the first commit makes the other proposal positive: $+0.0538203660$ or $+0.0759748183$, depending on the processing order. It is then rejected. These values come from controlled CPU replay of the acceptance and window-controller paths. They do not constitute a native-GPU concurrency test.

F Implementation and reproducibility

The CPU evaluator, tree encoders, and full-tree optimizer are implemented in `phylo2vec_fastme.c`, with input validation and Python bindings in `discrete_gradient.py`. The lower-triangular and full-coordinate interfaces require an ordered baseline. The multi-GPU evaluator and its tiled variants are implemented in `phylo2vec_cuda_8xh100.cu`. Sequence refinement is exercised by `bench_rnasim_gpu.cu`. The distinct distance-oracle recovery workload is implemented in `bench_minibatch_recovery.cu`.

Independent checks include `verify_all.c` for objective identities and encoding boundaries, `verify_spr_coverage.py` for neighborhood enumeration, and `compare_with_fastme_cli.py` for the external reference. The profile-completeness and window-composition constructions are recorded with their reproduction commands in the discrete-gradient review artifacts. CPU-emulated tests exercise the window controller, complete component-weight arithmetic, and a shared-input batching path. Separate native CUDA comparisons of the later split-launch implementation and its final-tree hashes are recorded in `GEMINI_REPORT_REFINE_KERNEL.md` and the artifacts under `handoff_refine_kernel/`. These tests cover different execution paths and are identified separately in their records.

The archived sequence measurements in Appendix D are preserved in `paper/restructure_chatgpt/original.tex`. The later kernel report measures additional implementation optimizations, whose timings have not been substituted into that archived comparison. The sequence records use approximate proposals with full-objective acceptance. They therefore do not supply an end-to-end benchmark with exact condensed distances throughout candidate selection.

F.1 Search policies and parameters

The procedures in Algorithms 1 and 2 specify the search at the level of trees, coordinate encodings, and component distances. The following policies instantiate their scheduling and candidate-selection steps. The score L always uses component identities restored by the recorded permutation.

CPU block schedule. Full-tree search uses a single block with $K = N$ singleton components, so $G = T$ and $M = D$. For requested size $3 \le K_\star < N$, the CPU minibatch implementation exposes $w = \min\{K_\star - 2, \max\{4, \lfloor 2K_\star/3 \rfloor\}\}$ consecutive leaves in the current depth-first leaf order. Block b starts at offset $b \max\{1, \lfloor w/2 \rfloor\}$ modulo N. Maximal inactive subtrees become fixed components. The largest such component is split while the skeleton has fewer than $K_\star$ leaves and a further split is possible. Thus $K_\star$ is a construction target, while K is the resulting skeleton size. The leaf order and components are reconstructed from the updated tree for every block. A hierarchical schedule instead combines a global backbone, local component windows, and staggered or larger windows crossing preceding component boundaries. Regardless of schedule, the sequential driver constructs each block's outside information from the current tree.

Ordered representations and row selection. Order produces an ordered encoding and its component permutation by the construction in Appendix B. Normalise invokes this construction only when the current encoding contains a value $v_i > i$, composing the new map with π. Decode(v, π) restores original component identities and suppresses the artificial root. For each row, RankMin selects $j_i \in \arg\min_{0 \le j \le 2i} \Delta_{i,j}$, keeps rows with $\Delta_{i,j_i} < -\eta$, and sorts them by increasing change. The CPU policy uses deterministic traversal and row order to resolve ties. It uses $I = 500$ and $\epsilon(\ell) = 10^{-12}(|\ell| + 1)$.

Joint batch acceptance. Let $d_{\min} < 0$ be the best single change. BatchStep greedily packs at most eight rows, excluding shared marked pruned-subtree nodes and attachment endpoints. After the first row, it retains only rows with $\Delta_{i,j_i} < 0.05 d_{\min}$. It tests a batch only if its predicted changes sum to less than $1.05 d_{\min}$. Every trial applies the selected replacements to the

same pre-trial encoding v and evaluates the complete candidate v'. A batch is accepted when $L(T(v'); M_\pi) < \ell + d_{\min} - \eta$. If rejected, the retained ranked prefix is halved and evaluated again, down to two moves. If no batch is accepted, the best single replacement is used. The predicted sum screens a batch but does not determine its acceptance.

SPR completion and deterministic escapes. When no improving coordinate row remains, the CPU solver first tests additional regraft edges for its already-scanned pruned components. If this fails, it tries the reversed ordered labeling. Only after both stages fail does it scan the remaining unrooted SPR moves, including complement-side prunes. ExtraSPR and RemainingSPR return an updated encoding in the current component labeling, or $\perp$ when no change is below $-\eta$.

Following an unsuccessful completion scan, Escape permits at most $E = 2$ attempts, each with at least one subsequent sweep available. For each row it retains the smallest change strictly between η and 0.05, then sorts these candidates by increasing change. Attempt $e \in \{1, 2\}$ combines the first and $(e + 1)$th candidates if both exist and their predicted changes sum to less than 0.02. Otherwise it returns $\perp$. These thresholds are in objective units. An escape may increase the working score. The return rule compares the final trial with the retained best scored tree. Consequently, a completed scan and a budget-limited exit represent different stopping conditions.

GPU skeleton solver. The GPU policy performs a bounded sequence of trial updates, packs SPR moves whose marked paths are disjoint, and rescores each combined tree on the next sweep. It retains the best scored skeleton and tries the reversed labeling when no improving row remains. It does not use the CPU batch-halving or full SPR completion steps. The best trial is compared with the entry skeleton before committing the block. Intermediate trial scores can increase, so only the retained best score and accepted block changes have a monotonicity interpretation. For the approximate-proposal sequence variant, the acceptance conditions and measurement scope are given in Appendix D.

G Supplementary CPU comparisons

Tables 5 and 6 give the full Apple M5 comparisons. FME denotes the in-memory reference implementation. P2V denotes Phylo2Vec. RF denotes normalized Robinson–Foulds distance. Wins and ties refer to the BME objective, with tolerance 10^{-9}. Timings and evaluation counts are means, with standard deviations and the reported P2V median and interquartile range retained.

Table 5: Additive distance comparison on Apple M5.

Objective and topology

N	FME L	P2V L	Win/tie/loss	FME RF	P2V RF
20	11.12739	11.12739	**0 / 20 / 0**	0.0000	0.0000
50	29.42572	29.42572	**0 / 20 / 0**	0.0000	0.0000
100	59.52693	59.52693	**0 / 10 / 0**	0.0000	0.0000
150	89.04365	89.04365	**0 / 5 / 0**	0.0000	0.0000
200	116.32398	116.32398	**0 / 5 / 0**	0.0000	0.0000

Computational cost

N	FME iters.	P2V evals.	FME ms	P2V ms	P2V med. [IQR]	Ratio
20	13.4	11.2	0.12 ± 0.02	0.08 ± 0.01	0.07 [0.02]	**1.54**×
50	42.2	19.4	1.58 ± 0.35	0.57 ± 0.13	0.59 [0.16]	**2.78**×
100	97.7	35.4	9.05 ± 0.84	2.63 ± 0.26	2.65 [0.44]	**3.44**×
150	156.2	49.4	31.47 ± 0.77	7.92 ± 0.86	7.75 [0.64]	**3.97**×
200	211.0	63.6	75.46 ± 2.42	18.14 ± 0.74	18.05 [1.17]	**4.16**×

Table 6: Noisy distance comparison on Apple M5.

Objective and topology

N	FME L	P2V L	Win/tie/loss	FME RF	P2V RF
20	11.19777	11.19777	**0 / 20 / 0**	0.0500	0.0500
50	29.25507	29.25607	**1 / 18 / 1**	0.0660	0.0649
100	59.25471	59.25471	**0 / 10 / 0**	0.0526	0.0526
150	92.14846	92.14846	**0 / 5 / 0**	0.0395	0.0395
200	120.94220	120.94220	**1 / 4 / 0**	0.0467	0.0487

Computational cost

N	FME iters.	P2V evals.	FME ms	P2V ms	P2V med. [IQR]	Ratio
20	14.4	12.4	0.07 ± 0.01	0.05 ± 0.01	0.04 [0.02]	**1.60**×
50	46.1	25.9	1.07 ± 0.07	0.49 ± 0.09	0.49 [0.10]	**2.18**×
100	102.6	41.0	10.62 ± 2.53	3.32 ± 0.56	3.18 [0.52]	**3.19**×
150	166.0	61.0	33.97 ± 1.01	10.15 ± 1.35	10.17 [1.35]	**3.35**×
200	235.8	84.4	85.16 ± 2.06	24.67 ± 2.79	24.51 [4.48]	**3.45**×

H Supplementary scaling measurements

The tables below retain the reported sweep measurements, separating CPU/Metal, CUDA, sharded, and embedding-based workloads. The single-GPU speedups use their respective host CPU references. They should not be interpreted as comparisons against one common host baseline. Stage timings measure balanced-distance construction and SPR evaluation, rather than optimization to convergence. The $2N^2$ column in the sharded table is the original rectangular-workload size proxy. It is not the number of valid Phylo2Vec coordinate entries, which is $(N-1)^2$, nor a count of distinct neighboring trees. Reported memory units are retained from the benchmark records.

For the embedding workload, each batch has 20,000 skeleton leaves, comprising 15,000 expanded leaves and 5,000 condensed components. Its epoch columns multiply average condensation-plus-gradient time by $\lceil N/15{,}000 \rceil$. They are workload estimates rather than measured convergence times. The largest listed validation discrepancy in this workload is 1.4×10^{-5}, and the listed validation outcomes refer to the benchmark's own checks.

Table 7: CPU and Metal gradient timings on Apple M5. All times are milliseconds.

FP64 full sweep

N	1 CPU	10 CPU	Ratio	10 CPU tiled
100	0.055	0.068	0.80×	0.096
250	0.332	0.322	1.03×	0.246
500	1.273	0.433	**2.94**×	0.508
1,000	5.860	1.878	**3.12**×	2.081
2,000	27.399	10.230	**2.68**×	10.806
3,000	85.815	24.428	**3.51**×	**23.777 ms**
4,000	132.225	51.265	**2.58**×	**43.329 ms**

Balanced-distance construction

N	CPU FP64	CPU FP32	Metal FP32	Ratio
100	0.012	0.011	0.517	0.02×
250	0.076	0.058	0.906	0.08×
500	0.312	0.223	1.216	0.26×
1,000	1.565	1.344	2.699	0.58×
2,000	8.954	9.533	4.462	**2.01**×
3,000	22.504	35.030	8.970	**2.51**×
4,000	68.188	59.289	7.571	**9.01**×

Hybrid sweep and absolute errors

N	Hybrid FP32	Ratio	FP64 error	FP32 error
100	0.524	0.10×	0.00e+00	4.35e-07
250	0.934	0.36×	0.00e+00	5.24e-07
500	1.742	0.73×	5.55e-17	6.63e-07
1,000	3.097	**1.89**×	4.44e-16	8.11e-07
2,000	6.312	**4.34**×	4.44e-16	8.93e-07
3,000	14.359	**5.98**×	5.27e-16	9.84e-07
4,000	26.922	**4.91**×	5.00e-16	1.13e-06

Table 8: Single-GPU gradient timings and agreement with the CPU reference. Times are milliseconds.

RTX A6000 timings and reported maximum errors

N	A6000 full	Ratio	A6000 tiled	Max. error
100	0.061	**8.44**×	0.078	3.68e-07
250	0.186	**15.92**×	0.207	6.22e-07
500	0.382	**32.95**×	0.511	6.48e-07
1,000	1.035	**33.69**×	1.988	9.29e-07
2,000	3.851	**40.15**×	7.288	1.11e-06
4,000	11.053	**62.48**×	43.124	1.07e-06

H100 timings

N	CPU ref.	H100 avg.	H100 SPR	H100 total	Ratio	H100 tiled
100	0.249	0.044	0.027	0.071	**3.51**×	0.095
250	1.964	0.116	0.048	0.164	**11.98**×	0.205
500	8.176	0.227	0.077	0.304	**26.89**×	0.460
1,000	32.336	0.661	0.105	0.766	**42.21**×	1.030
2,000	154.091	1.557	0.726	2.283	**67.50**×	4.273
4,000	701.375	3.565	2.654	6.219	**112.78**×	22.327

The maximum-error column is the reported combined check across the GPU paths. All listed checks pass their benchmark tolerance. CPU baselines differ between hosts.

Table 9: Full-table and tiled multi-GPU sweeps. Times are milliseconds and memory is reported in MB.

A6000 timings and memory columns

N	2 A6000 full	2 A6000 tiled	Full memory	Tiled memory
1,000	0.93	5.75	27	39
2,000	4.89	13.90	92	93
4,000	14.69	56.85	336	247
10,000	90.45	249.59	1,985	1,076
20,000	422.73	1,212.27	7,785	3,677
40,000	*OOM*	5,628.62	31,142	13,614

Full-table H100 sweep

N	1 H100 [avg.+SPR]	8 H100	Ratio
1,000	0.76 [0.65 + 0.11]	0.79	0.96×
2,000	2.21 [1.47 + 0.74]	1.71	**1.29**×
4,000	6.01 [3.57 + 2.44]	3.98	**1.51**×
10,000	29.61 [10.10 + 19.50]	13.03	**2.27**×
20,000	155.93 [23.23 + 132.70]	41.06	**3.80**×
40,000	816.34 [60.08 + 756.26]	**158.36**	**5.15**×

Tiled H100 sweep

N	1 H100 tiled	8 H100 tiled	Top-1 check
1,000	1.46	4.47	**PASS** (0.00e+00)
2,000	6.14	5.21	**PASS** (2.38e-07)
4,000	30.78	10.98	**PASS** (0.00e+00)
10,000	250.69	57.53	**PASS** (0.00e+00)
20,000	1,123.43	244.13	**PASS** (4.77e-07)
40,000	4,768.31	**1,032.83**	**PASS** (0.00e+00)

Table 10: Distributed sweeps on eight H100 GPUs.

Timing

N	Work proxy $2N^2$	Distance ms	SPR ms	Total s
40,000	3.20×10^9	514.54	517.92	**1.032**
80,000	1.28×10^{10}	2,112.82	2,321.28	**4.434**
120,000	2.88×10^{10}	4,785.75	4,803.60	**9.589**
180,000	6.48×10^{10}	10,848.67	13,977.73	**24.826**
250,000	1.25×10^{11}	39,981.55	43,721.48	**83.703**
350,000	2.45×10^{11}	76,240.05	96,515.83	**172.756**
380,000	2.89×10^{11}	89,992.00	115,457.57	**205.450**

Memory and execution mode

N	MB/GPU	Aggregate GB	Mode and check
40,000	2,386	18.6	`Full-Shard` [**PASS**]
80,000	7,824	61.1	`Shard+Calib` [**PASS**]
120,000	16,314	127.5	`Shard+Calib` [**PASS**]
180,000	34,770	271.6	`Shard+Calib` [**PASS**]
250,000	62,296	486.7	`Shard+Calib` [**PASS**]
350,000	62,180	485.8	`StreamD+Calib` [**PASS**]
380,000	72,946	569.9	`StreamD+Calib` [**PASS**]

At $N = 400{,}000$, the reported requirement is 80,600 MB/GPU (629.7 GB aggregate), exceeding available device memory. Full-Shard and Shard+Calib retain distance shards. StreamD+Calib generates distance tiles as required. The work proxy is not a count of distinct moves.

Table 11: Embedding-based condensed sweeps with $K = 20{,}000$ skeleton leaves. The epoch columns are batch-count times mean condensation-plus-gradient time.

Two A6000 GPUs

N	Batches	2 A6000 cond.+grad. (ms)	Epoch (s)
1,000,000	67	$4.45 + 284.44$	19.36
5,000,000	334	$8.42 + 283.02$	97.34
10,000,000	667	$17.29 + 281.58$	199.35

Eight H100 GPUs

N	Cond. ms	Grad. ms	Epoch s	MB/GPU	Check
1,000,000	2.69	48.53	**3.43**	7,793	**PASS** (1.4e-05)
5,000,000	4.86	48.48	**17.82**	7,824	**PASS** (2.1e-06)
10,000,000	10.20	48.57	**39.20**	7,862	**PASS** (2.2e-06)

This workload uses a bilinear embedding distance and is separate from sequence-distance refinement. The validation figures are observed discrepancies, not universal error bounds.